Predicting Calorie Expenditure During Exercise

Keith Fernandes
fernandes.kei@northeastern.edu

Shreyans Jain
jain.shreyans@northeastern.edu

GitHub: https://github.com/keithfernandes12/Predict_Calorie_Expenditure

Abstract
We present a stacking ensemble approach to predict calorie expenditure during exercise for the Kaggle Playground Series S5E5 competition. Ten diverse base learners, gradient boosting frameworks (LightGBM gbdt/goss, XGBoost, CatBoost, HistGradientBoosting), bagging ensembles (Random Forest, Extra Trees), Google's Yggdrasil Decision Forests, AutoGluon, and a Keras neural network, they are trained under identical 5-fold cross-validation on 750,000 rows augmented with 15,000 rows from the original Calories Burnt Prediction dataset. A Ridge regression meta-learner trained on out-of-fold (OOF) predictions combines these base models, achieving a final CV RMSLE of 0.059159 (RMSE ≈ 3.56 calories), outperforming every individual base model. We also ablate the contribution of two engineered features (BMI and Duration × Heart_Rate) and discuss the design choices behind the stacking protocol.

1. Introduction

Estimating calorie expenditure from physiological and workout data has practical applications in fitness tracking, personalized health coaching, and clinical monitoring. The Kaggle Playground Series S5E5 competition frames this as a tabular regression problem: given measurements for age, sex, height, weight, workout duration, heart rate, and body temperature, predict total calories burned.
The competition metric is Root Mean Squared Logarithmic Error (RMSLE):

$$\text{RMSLE} = \left(\frac{1}{n}\sum_{i=1}^{n}(\log(1+\hat{y}_i) - \log(1+y_i))^2\right)^{\frac{1}{2}}$$

RMSLE penalizes relative rather than absolute errors, making it appropriate for a target that spans a wide range and should not be dominated by high-calorie outliers.
Our approach is a multi-model stacking ensemble: train heterogeneous base learners under a shared cross-validation protocol, then train a Ridge meta-learner on the OOF predictions from each base model. This design avoids target leakage, leverages the complementary strengths of diverse model families, and uses L2 regularization to handle the collinearity that is inherent in stacking.

2. Dataset

2.1. Sources

Per competition rules, the original dataset may be freely incorporated into training. Because it was generated by a deep learning model trained on the original data, the Kaggle datasets share the same feature space but have slightly different distributions.

Dataset	Rows	Description
train.csv	750,000	Synthetically generated from the original dataset
test.csv	250,000	Labels withheld; predictions submitted to Kaggle
calories.csv	15,000	Original real-world Calories Burnt Prediction dataset

2.2. Features

Feature	Type	Range	Description
Sex	Binary	{0, 1}	male → 0, female → 1
Age	Continuous	20–79	Age in years
Height	Continuous	126–222 cm	Height
Weight	Continuous	36–132 kg	Body weight
Duration	Continuous	1–30 min	Workout duration
Heart_Rate	Continuous	67–128 bpm	Average heart rate during workout
Body_Temp	Continuous	37.1–41.5 °C	Body temperature during workout

Target: Calories-continuous, right-skewed.

2.3. Data Quality

No features contain missing values. Body_Temp exhibits 14,919 IQR-based outliers (~2% of rows), likely representing extreme physiological readings during intense exercise. Height and Weight have a small number of outliers each. Because tree-based models are inherently robust to outliers and the synthetic generation process may naturally produce such values, no imputation or clipping was applied.

Feature	Missing	IQR Outliers	Mean	Std
Sex	0	0	0.50	0.50
Age	0	0	41.42 yr	15.18
Height	0	14	174.70 cm	12.82
Weight	0	9	75.15 kg	13.98
Duration	0	0	15.42 min	8.35
Heart_Rate	0	36	95.48 bpm	9.45
Body_Temp	0	14,919	40.04 °C	0.78

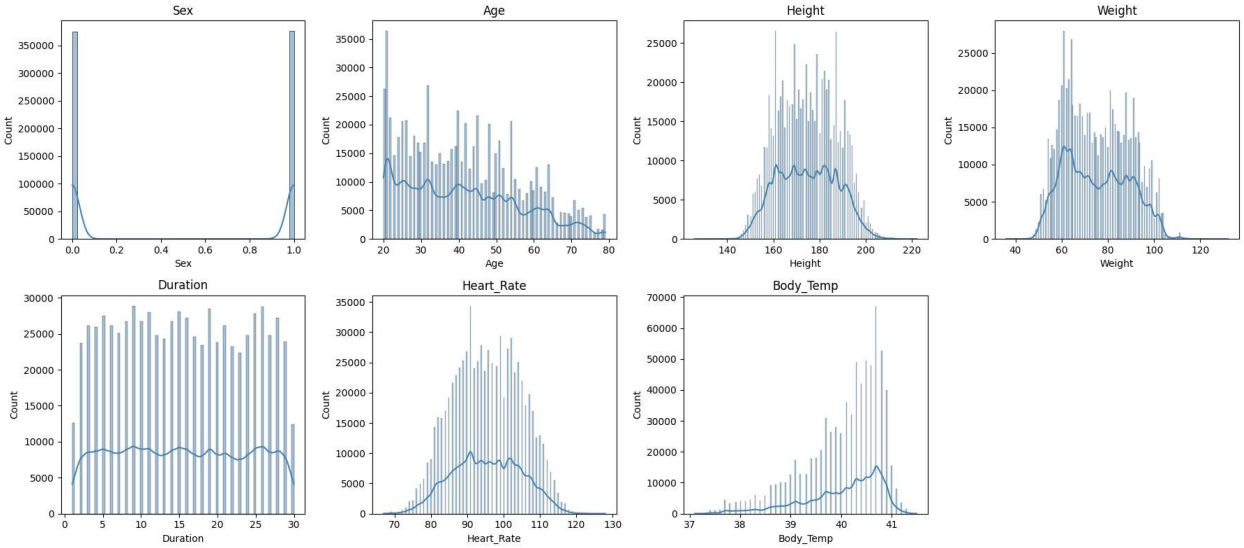

Figure 1: *Feature distributions across the training set.*

3. Exploratory Data Analysis

3.1. Target Distribution and Log Transformation

The target Calories is right-skewed. We apply np.log1p(Calories) before training, which (1) reduces skew, improving model fit, and (2) makes RMSE on the transformed target mathematically equivalent to RMSLE on the original target, meaning all base models directly optimize the evaluation metric. Predictions are back-transformed with np.expm1() at submission time.

3.2. Mutual Information

Mutual information (MI) was computed between each feature (including engineered features) and the log-transformed target to assess non-linear predictive relevance:

Feature	Mutual Information
Duration × Heart_Rate *(engineered)*	1.9911
Duration	1.6411
Body_Temp	1.1208
Heart_Rate	0.9779
BMI *(engineered)*	0.1113
Age	0.0976
Weight	0.0561
Height	0.0551
Sex	0.0162

The engineered Duration × Heart_Rate interaction term is the single most informative variable, confirming that cardiovascular load (intensity × time) is a stronger predictor of calorie burn than either duration or heart rate alone. Body temperature, which reflects physiological response to exertion, ranks third among raw features.

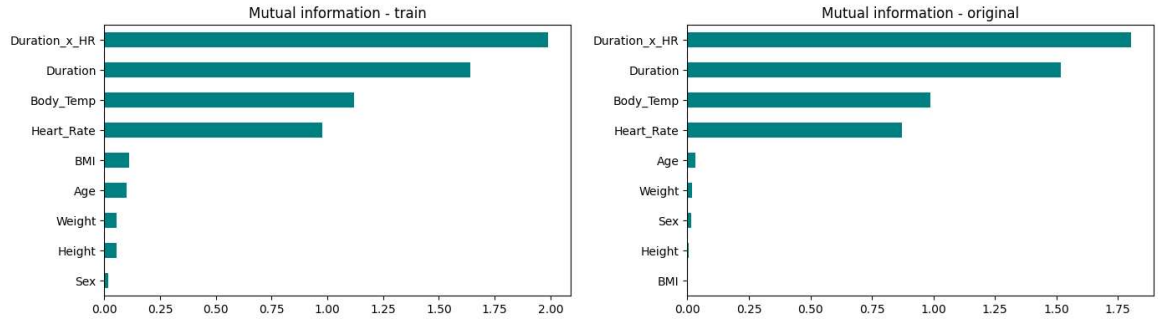

Figure 2: *Mutual information scores for the Kaggle training set (left) and the original dataset (right).*

3.3. Correlation Structure

Correlation heatmaps for both the Kaggle training set and the original dataset show strong positive correlations between Duration, Heart_Rate, Body_Temp, and the target. Weight and Height show moderate mutual correlation, as expected, but weak direct correlation with calorie expenditure. The relatively simple linear structure suggests that most predictive power comes from exercise-intensity variables rather than static body measurements.

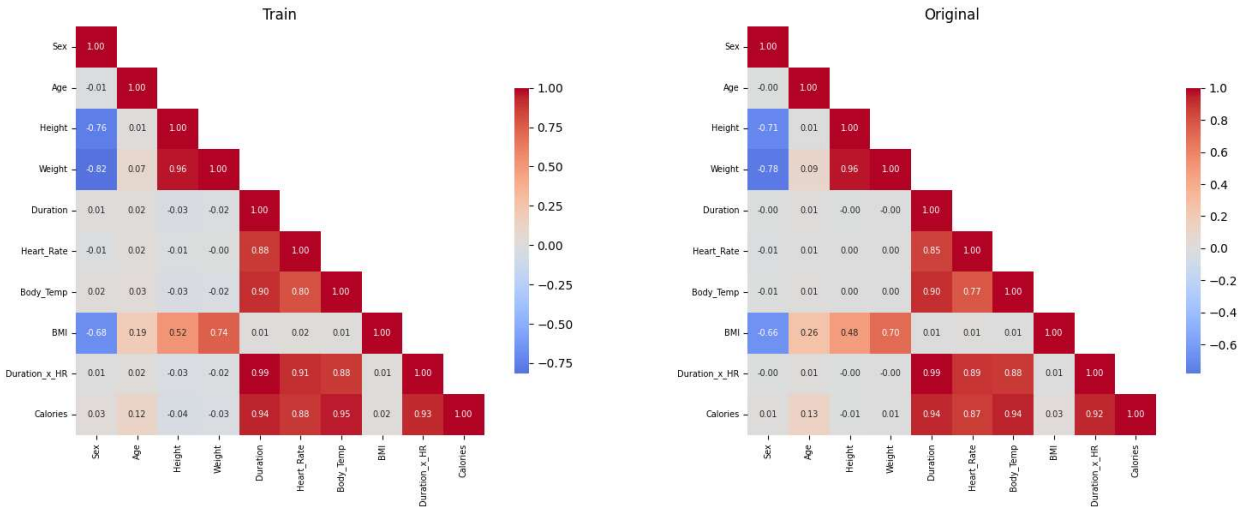

Figure 3: *Pairwise correlation heatmaps for the Kaggle training set (left) and the original dataset (right).*

4. Methodology

4.1. Feature Engineering

Two features were engineered from domain knowledge and applied identically to the training set, test set, and external dataset:

1. **BMI** = Weight / (Height / 100)², a standard body composition index that captures the relationship between weight and height more compactly than either column alone.
2. **Duration × Heart_Rate**, a proxy for total cardiovascular load (exercise intensity × time). This interaction is the most informative variable in the dataset according to mutual information analysis.

The final feature set is 9 columns: the 7 original features plus BMI and Duration × Heart_Rate.

4.2. Training Protocol

All base models share a common training protocol to ensure fair comparison:

- **Cross-validation:** 5-fold KFold(shuffle=True, random_state=42)
- **External data augmentation:** The 15,000-row original dataset is appended to the training split of each fold. It is never added to the validation split, preventing leakage.
- **Metric:** RMSLE (computed as RMSE on log1p(y))

For each model, the trainer produces:

- **Per-fold RMSLE scores**
- **Out-of-fold (OOF) predictions** on the full 750,000-row training set
- **Test predictions** averaged across all 5 folds

4.3. Base Models

Ten base models were trained spanning four model families:

Gradient Boosting:

Model	Key Hyperparameters
HistGradientBoosting	lr=0.0117, max_depth=59, max_iter=4,454, l2_regularization=10.4
LightGBM (gbdt)	lr=0.060, num_leaves=89, reg_λ=87.3, n_estimators=50,000 + early stopping
LightGBM (goss)	lr=0.065, num_leaves=13, reg_λ=10.8, n_estimators=50,000 + early stopping
XGBoost	lr=0.037, max_depth=11, min_child_weight=96, n_estimators=50,000 + early stopping
CatBoost	lr=0.031, depth=8, l2_leaf_reg=6.1, iterations=50,000 + early stopping

Tree Ensembles (Bagging):

Model	Key Hyperparameters
Random Forest	n_estimators=300, min_samples_leaf=5
Extra Trees	n_estimators=300, min_samples_leaf=5
Yggdrasil (YDF)	num_trees=1,000, max_depth=8

AutoML:

Model	Key Hyperparameters
AutoGluon	medium_quality preset, 600 s per fold

Neural Network:

Model	Architecture & Training
Keras ANN	Dense(256, SELU) → Dense(128, SELU) → Dense(64, SELU) → Dense(1); AdamW (lr=1e-3); EarlyStopping (patience=7); ReduceLROnPlateau (patience=3, ×0.3); bagged over 3 random seeds per fold

Gradient boosting hyperparameters (HistGB, LightGBM, XGBoost, CatBoost) were tuned in prior Optuna sessions and fixed for this run. The Keras network uses per-fold StandardScaler (fit only on the training split) to normalize inputs without leakage.

4.4. Stacking with Ridge Regression

After base model training, OOF predictions are stacked column-wise into a meta-feature matrix of shape (750,000 × 10). A Ridge regression meta-learner is trained on this matrix to learn an optimal, data-driven combination of base model predictions.

Why Ridge instead of ordinary least squares?

OOF predictions from 10 models all targeting the same variable are highly collinear. Ordinary least squares would assign unstable, large-magnitude coefficients that could cause any single base model to dominate due to multicollinearity. Ridge's L2 penalty regularizes the coefficients toward zero, producing a stable weighted combination that generalizes to unseen data.

Why OOF predictions instead of training predictions?

Using each model's predictions on its own training data would allow the meta-learner to favor whichever model memorized training examples best, rather than which model generalizes best. OOF predictions are made on held-out data, data the base model has never seen, so the meta-learner only observes genuinely out-of-sample signals.

4.5. Ridge Hyperparameter Tuning (Optuna)

Ridge hyperparameters were tuned using Optuna with 250 trials and a TPE sampler (multivariate=True):

Parameter	Search Space	Optimized Value
alpha (L2 strength)	[0, 10]	~9.999
tol (convergence tolerance)	[1e-6, 1e-2]	~8.6e-3

The best alpha converging to the upper bound of the search space (9.999) is a signal that the true optimum may lie above 10, widening the search to [0, 100] is recommended in future work.

5. Results

5.1. Model Benchmark

All metrics are computed on OOF predictions under the same 5-fold CV protocol. RMSE (calories) is the calorie-scale RMSE after expm1 back-transformation and is reported as a secondary, interpretable metric.

Model	RMSLE (mean ± std)	RMSE (calories)
Ridge Ensemble	**0.059159 ± 0.000384**	**3.5631**
AutoGluon	0.059403 ± 0.000376	3.5846
LightGBM (gbdt)	0.059754 ± 0.000311	3.6061
LightGBM (goss)	0.059778 ± 0.000361	3.5959
CatBoost	0.059793 ± 0.000334	3.6423
HistGB	0.059890 ± 0.000434	3.6293
Yggdrasil	0.060398 ± 0.000336	3.6796
KerasANN	0.060609 ± 0.000624	3.6659
XGBoost	0.060708 ± 0.000480	3.7202
ExtraTrees	0.061240 ± 0.000556	3.7060
RandomForest	0.061803 ± 0.000643	3.7383

Key observations:

- The Ridge ensemble achieves **RMSLE = 0.059159**, improving over the best single model (AutoGluon, 0.059403) by 0.000244, a small but consistent gain that confirms stacking adds value beyond any individual model.
- **Gradient boosting dominates**: AutoGluon and the four GBM variants cluster within a 0.0005 RMSLE band (0.0594–0.0599), well ahead of the remaining models.
- **Bagging methods trail by ~0.002 RMSLE**: Random Forest and Extra Trees lag the leading models by ~0.002 RMSLE, consistent with bagging being weaker than boosting on smooth, low-noise regression targets.
- **Fold-level variance is low** (std ≈ 0.0003–0.0006 across all models): model rankings are stable across folds and are not an artifact of high variance.

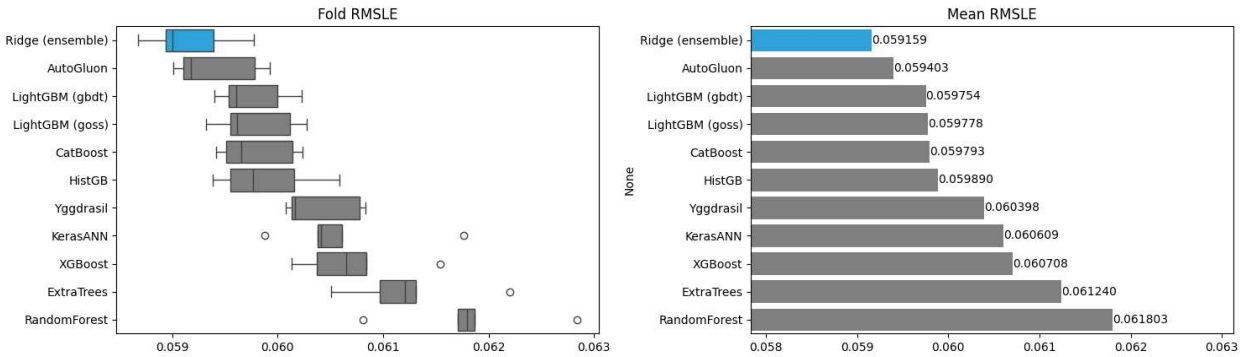

Figure 4: *Per-fold RMSLE boxplots (left) and mean RMSLE bar chart (right) for all models.*

5.2. Ablation: Effect of Engineered Features

To quantify the contribution of BMI and Duration × Heart_Rate, we re-run LightGBM (gbdt) under the same 5-fold protocol with and without the two engineered features:

Variant	Features	RMSLE (OOF)	RMSE (calories)
With BMI + Duration × Heart_Rate	9	0.059755	3.6061
Without (7 raw features only)	7	0.059790	3.6126
Δ (with-without)		**−0.000034**	**−0.0065**

The engineered features produce a marginal improvement (~0.06% relative RMSLE reduction) that falls well within the fold-level standard deviation (~0.0003). Tree ensembles can reconstruct BMI and Duration × Heart_Rate from the raw columns by learning appropriate splits, so the delta reflects improved convergence efficiency rather than new information. The features retain value for interpretability, particularly for mutual information analysis, where Duration × Heart_Rate ranks as the single most informative variable.

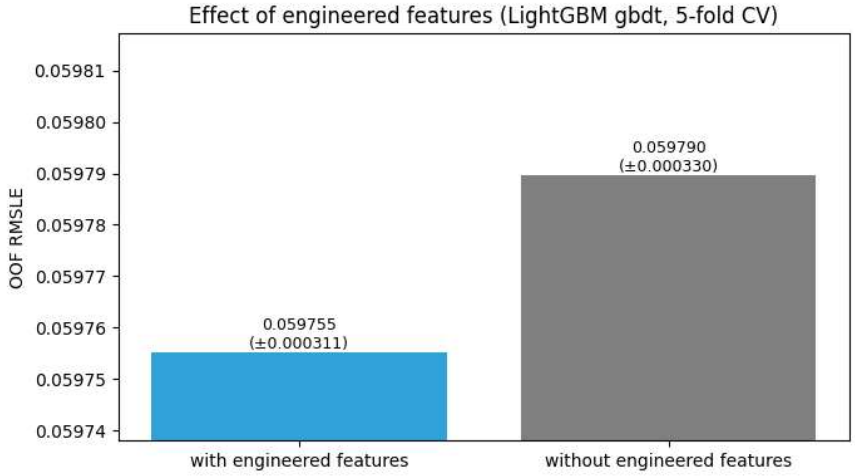

Figure 5: *OOF RMSLE comparison for LightGBM (gbdt) with and without engineered features.*

5.3. Ridge Meta-Learner Coefficients

After 5-fold training, Ridge coefficients are averaged across folds and visualized as a horizontal bar chart below. With alpha ≈ 10, all coefficients are regularized toward zero, no single base model dominates the ensemble. The coefficient magnitudes reflect each model's marginal contribution given the predictions of all other models, which in this setting is shaped primarily by how much unique, non-redundant signal each model contributes.

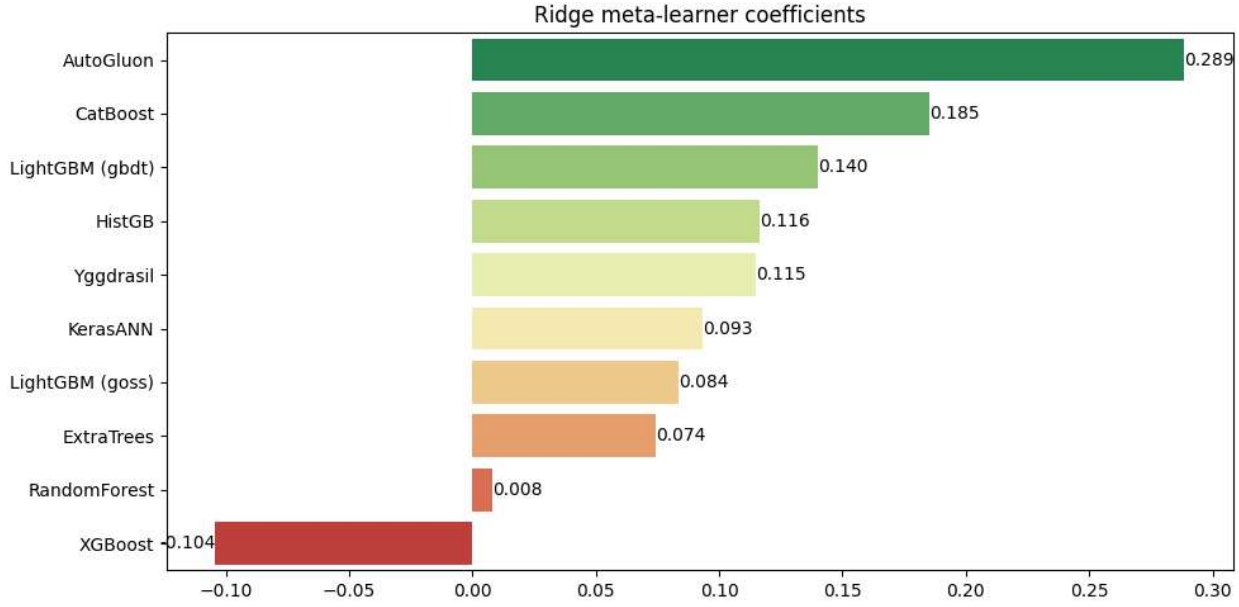

Figure 6: *Average Ridge meta-learner coefficients across 5 folds, sorted by magnitude.*

5.4. Final Submission

The final prediction pipeline:

1. Each base model's test predictions (averaged across 5 folds) form the meta-feature matrix.
2. Ridge ensemble predictions are generated from the 5-fold average of Ridge models.
3. np.expm1() back-transforms predictions from log-scale to calorie scale.
4. Submission file: sub_ridge_0.059161.csv.

Final CV RMSLE: 0.059161

6. Discussion

What worked

- **Stacking is effective.** The Ridge ensemble consistently outperformed all base models, even when the absolute gain is small (~0.000244 RMSLE over the best base model). This matches the theoretical expectation: a meta-learner trained on OOF predictions can find a weighted combination that exploits complementary errors among base models.
- **Log transformation is essential.** Transforming the target with log1p directly aligns the optimization objective (RMSE) with the evaluation metric (RMSLE). Without this, models would minimize calorie-scale error rather than log-scale error.
- **External data augmentation helps.** Appending 15,000 rows of the original dataset to each training fold provides cleaner in-distribution signal and is particularly valuable for the Keras network, which benefits from additional training examples in each fold.
- **Gradient boosting frameworks dominate.** The top five individual models are all boosting-based. Bagging methods (Random Forest, Extra Trees) trail by ~0.002 RMSLE, a consistent finding on smooth, well-sampled tabular regression targets.

Limitations and future directions

1. **Ridge alpha search space is too narrow.** The Optuna optimum saturated at the upper bound (~9.999). Widening the search range to [0, 100] or [0, 1000] is likely to find a better-calibrated regularization strength.
2. **Ablation covers only one model.** The feature engineering ablation was run on LightGBM alone. Neural networks are less able to reconstruct non-linear interactions from raw inputs, so BMI and Duration × Heart_Rate likely matter more to the Keras model, repeating the ablation on the NN would produce a more defensible claim.
3. **Bottom-tier models may add noise.** RandomForest and ExtraTrees contribute ~0.002 RMSLE worse signal than the top tier. Removing them from the stack and re-tuning Ridge could reduce noise while preserving ensemble diversity.
4. **Additional feature engineering is unexplored.** Heart rate reserve (HR_max − Heart_Rate), MET-style intensity proxies, and polynomial interaction terms between Age and Duration have not been evaluated.

7. Conclusion

We built a stacking ensemble for calorie expenditure prediction that trains ten diverse base learners under a shared 5-fold cross-validation protocol and combines them with a Ridge meta-learner trained on OOF predictions. The final ensemble achieves **CV RMSLE = 0.059159**, improving over the best individual model (AutoGluon, RMSLE = 0.059403) by approximately 0.25%. Gradient boosting frameworks dominate individual model performance, while the neural network and bagging ensembles contribute complementary diversity. Feature engineering (BMI and Duration × Heart_Rate) provides a marginal but interpretability-relevant improvement. All model artifacts, OOF predictions, figures, and the submission file are archived in Results/ for full reproducibility.

References

1. Reade, W. & Park, E. (2025). *Predict Calorie Expenditure*. Kaggle Playground Series S5E5. https://kaggle.com/competitions/playground-series-s5e5
2. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *NeurIPS 2017*.
3. Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *KDD 2016*.
4. Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased Boosting with Categorical Features. *NeurIPS 2018*.
5. Erickson, N., Mueller, J., Shirkov, A., Zhang, H., Larroy, P., Li, M., & Smola, A. (2020). AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data. *ICML AutoML Workshop*.
6. Guillame-Bert, M., Arnaud, C., Janin, R., & Batzner, S. (2022). Yggdrasil Decision Forests: A Fast and Extensible Decision Forests Library. *SIGKDD 2022*.
7. Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *KDD 2019*.
8. Wolpert, D. H. (1992). Stacked Generalization. *Neural Networks, 5*(2), 241–259.